

Retrieval pipeline setup for Resume using vibe coding. (Followed instructions to implement resumes-ai-rag)

Created 'resumes-ai-rag' repo to have endpoints like

GET /v1/health

GET /v1/health/db

POST /v1/embeddings

POST /v1/search/bm25

POST /v1/search/vector

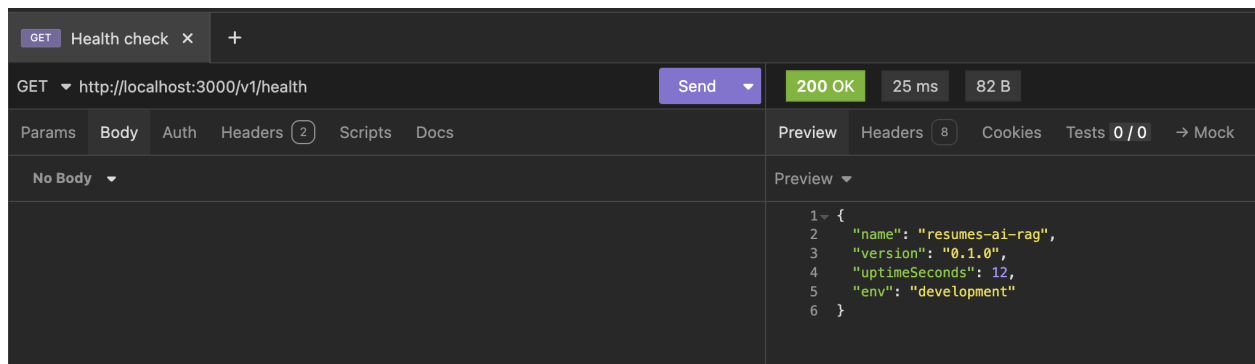
POST /v1/search/hybrid

POST /v1/search/rerank

POST /v1/search/summarize

POST /v1/search

GET /v1/health



GET Health check x +

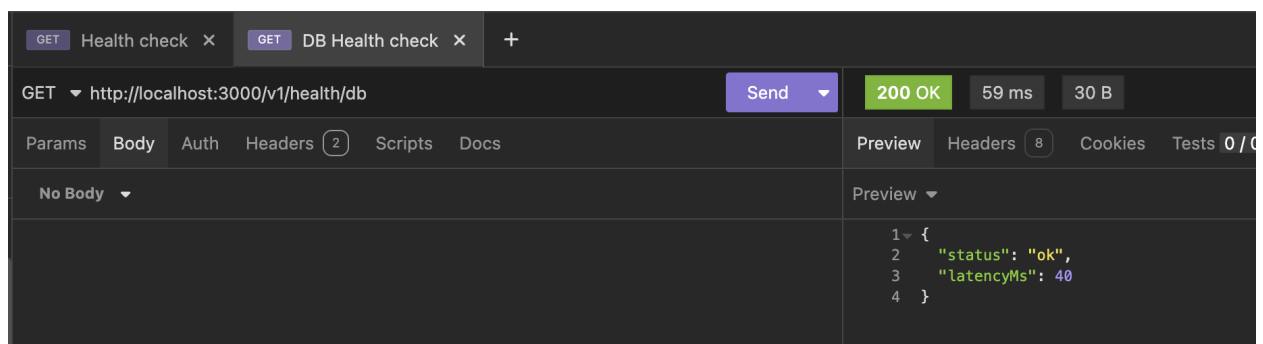
GET http://localhost:3000/v1/health Send 200 OK 25 ms 82 B

Params Body Auth Headers (2) Scripts Docs Preview Headers (8) Cookies Tests 0/0 → Mock

No Body

```
1 {
2   "name": "resumes-ai-rag",
3   "version": "0.1.0",
4   "uptimeSeconds": 12,
5   "env": "development"
6 }
```

GET /v1/health/db



GET Health check x GET DB Health check x +

GET http://localhost:3000/v1/health/db Send 200 OK 59 ms 30 B

Params Body Auth Headers (2) Scripts Docs Preview Headers (8) Cookies Tests 0/0

No Body

```
1 {
2   "status": "ok",
3   "latencyMs": 40
4 }
```

POST /v1/embeddings

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/v1/embeddings`. The status is **200 OK** with a response time of 542 ms and a body size of 19 KB. The request body is a JSON object: `{ "input": "hello world" }`. The response body is a JSON object: `{ "embedding": [ -0.01953125, 0.0267486572265625, 0.012451171875, 0.028839111328125, 0.04248046875, 0.04327392578125, 0.012847900390625, 0.025299072265625, -0.022552490234375, -0.00878143310546875, -0.02215576171875, 0.0290985107421875, -0.0205841064453125, -0.034088134765625, -0.04433153319375 ] }`.

```
POST http://localhost:3000/v1/embeddings 200 OK 542 ms 19 KB
```

Params Body Auth Headers (3) Scripts Docs Preview Headers (8) Cookies Tests 0/0 → Mock C

JSON Preview

```
1 { "input": "hello world" }
```

```
1 {  
2   "embedding": [  
3     -0.01953125,  
4     0.0267486572265625,  
5     0.012451171875,  
6     0.028839111328125,  
7     0.04248046875,  
8     0.04327392578125,  
9     0.012847900390625,  
10    0.025299072265625,  
11    -0.022552490234375,  
12    -0.00878143310546875,  
13    -0.02215576171875,  
14    0.0290985107421875,  
15    -0.0205841064453125,  
16    -0.034088134765625,  
17    -0.04433153319375  
2   ]  
}
```

POST /v1/search/bm25

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/v1/search/bm25`. The status is **200 OK** with a response time of 61 ms and a body size of 28 B. The request body is a JSON object: `{ "query": "automation QA engineer", "topK": 10, "filters": { "minYearsExperience": 2 } }`. The response body is a JSON object: `{ "results": [], "searchMs": 44 }`.

```
POST http://localhost:3000/v1/search/bm25 200 OK 61 ms 28 B
```

Params Body Auth Headers (3) Scripts Docs Preview Headers (8) Cookies Tests 0/0 → Mock Console

JSON Preview

```
1 { "query": "automation QA engineer", "topK": 10, "filters":  
  { "minYearsExperience": 2 } }
```

```
1 {  
2   "results": [],  
3   "searchMs": 44  
4 }
```

POST /v1/search/vector

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/v1/search/vector`. The status is **200 OK** with a response time of 724 ms and a body size of 29 B. The request body is a JSON object: `{ "query": "automation QA engineer", "topK": 10, "filters": { "minYearsExperience": 1 } }`. The response body is a JSON object: `{ "results": [], "searchMs": 702 }`.

```
POST http://localhost:3000/v1/search/vector 200 OK 724 ms 29 B
```

Params Body Auth Headers (3) Scripts Docs Preview Headers (8) Cookies Tests 0/0 → Mock C

JSON Preview

```
1 { "query": "automation QA engineer", "topK": 10, "filters": { "minYearsExperience": 1 } }
```

```
1 {  
2   "results": [],  
3   "searchMs": 702  
4 }
```

POST /v1/search/hybrid

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/v1/search/hybrid`. The status is **200 OK** with a response time of 497 ms and a body size of 50 B. The request body is a JSON object: `{ "query": "Marketing and Operations Enthusiast", "topK": 10, "filters": { "minYearsExperience": 1 } }`. The response body is a JSON object: `{ "results": { "bm25": [], "vector": [] }, "searchMs": 477 }`.

```
POST http://localhost:3000/v1/search/hybrid 200 OK 497 ms 50 B
```

Params Body Auth Headers (3) Scripts Docs Preview Headers (8) Cookies Tests 0/0 → Mock

JSON Preview

```
1 { "query": "Marketing and Operations Enthusiast", "topK": 10, "filters":  
  { "minYearsExperience": 1 } }
```

```
1 {  
2   "results": {  
3     "bm25": [],  
4     "vector": []  
5   },  
6   "searchMs": 477  
7 }
```

POST /v1/search/rerank

POSTrerankx+

POSThttp://localhost:3000/v1/search/rerankSend200 OK914 ms764 BJust Now

ParamsBodyAuthHeaders3ScriptsDocs

PreviewHeaders8CookiesTests0/0MockConsole

JSON

```
1- {
2  "query": "automation QA engineer",
3  "topK": 5,
4  "candidates": [
5    { "resumeId": "1", "snippet": "Experienced Automation QA with Selenium and Jenkins", "score": 0.8 },
6    { "resumeId": "2", "snippet": "Frontend engineer with React and CSS", "score": 0.1 },
7    { "resumeId": "3", "snippet": "QA engineer with API automation and Postman", "score": 0.7 }
8  ]
9 }
```

Preview

```
1- {
2  "results": [
3    {
4      "resumeId": "1",
5      "rank": 1,
6      "score": 0.9,
7      "reason": "Direct match with automation QA engineer experience and tools like Selenium and Jenkins",
8      "candidate": {
9        "resumeId": "1",
10       "snippet": "Experienced Automation QA with Selenium and Jenkins",
11       "score": 0.8
12     }
13   },
14   {
15     "resumeId": "3",
16     "rank": 2,
17     "score": 0.6,
18     "reason": "Relevant QA engineer experience with API automation, though not explicitly mentioning automation QA engineer",
19     "candidate": {
20       "resumeId": "3",
21       "snippet": "QA engineer with API automation and Postman",
22       "score": 0.7
23     }
24   },
25   {
26     "resumeId": "2",
27     "rank": 3,
28     "score": 0,
29     "reason": "No relevant experience in QA or automation, frontend engineer with React and CSS",
30     "candidate": {
31       "resumeId": "2",
32       "snippet": "Frontend engineer with React and CSS",
33       "score": 0.1
34     }
35   }
36 ],
37 "fallbackUsed": false,
38 "rerankMs": 892
39 }
```

POST /v1/search/summarize

POSTsummarizex+

POSThttp://localhost:3000/v1/search/summarizeSend200 OK350 ms233 BJust Now

ParamsBodyAuthHeaders3ScriptsDocs

PreviewHeaders8CookiesTests0/0MockConsole

JSON

```
1- {
2  "query": "senior Node.js backend engineer with MongoDB experience",
3  "candidate": { "resumeId": "abc", "snippet": "Experienced backend engineer with Node.js, Express, and MongoDB, built scalable APIs..." },
4  "style": "short",
5  "maxTokens": 150
6 }
```

Preview

```
1- {
2  "resumeId": "abc",
3  "style": "short",
4  "summary": "Senior Node.js backend engineer with hands-on experience in MongoDB, having built scalable APIs using Node.js and Express, making abc a strong fit.",
5  "fallbackUsed": false,
6  "summarizeMs": 333
7 }
```

POST /v1/search

POSTsearch x +

POST http://localhost:3000/v1/searchSend200 OK806 ms291 B

ParamsBodyAuthHeaders3ScriptsDocs

PreviewHeaders8CookiesTests0/0MockConsole

JSON

```
1 {
2   "query": "Marketing and Operations Enthusiast",
3   "topK": 20,
4   "filters": {"minYearsExperience": 2},
5   "options": {"rerankTopN": 8, "summarize": true, "summarizeTopK": 3}
6 }
```

Preview

```
1 {
2   "query": "Marketing and Operations Enthusiast",
3   "results": [],
4   "bm25Results": [],
5   "vectorResults": [],
6   "mergedCandidates": [],
7   "rerankMs": 0,
8   "bm25Ms": 43,
9   "vectorMs": 740,
10  "summarizeMs": 0,
11  "fallbackUsed": false,
12  "bm25Fallback": false,
13  "vectorFallback": false,
14  "rerankFallback": false,
15  "warnings": [],
16  "searchMs": 784
17 }
```